produces:

```
and:1 is:3 line:3 on:1 one:1 so:1 this:3 three:1 two:1
```

Same logic, same result. This time, though, we start the counter first, it creates its hash and then blocks at Ractor.receive. Then we create the reading ractor, which takes the counter as an argument, again because otherwise the ractor block would be isolated. Inside the block, the reader opens the files and scans as before, but this time it uses send to pass each word back to the counter without blocking. Note that in this case, we need to explicitly also send nil at the end to terminate the loop.

The final two lines are the same, waiting on the counter to be finished, but overall, this version of the code should block less and should allow the reader to get ahead of the worker if it can.

How Ractors Pass Variables

We've hinted variables passed to and from ractors don't behave the same way as regular variable passing does in Ruby. The goal of the ractor implementation is to prevent ractors from changing values that other ractors depend on. One way that is done is by preventing ractors from having access to mutable variables that exist outside the ractor scope.

As we've already mentioned, ractors are isolated from the rest of their binding, variables that would normally be in-scope for the block are not available inside the ractor.

Additionally, ractors apply special semantics to values that are passed to a ractor using send or yield. The ractor world divides Ruby objects into "shareable" or "unsharable."

Broadly, sharable objects are objects whose value can not be changed—immutable objects and objects that have been frozen. Specifically, the following are all considered shareable:

- The special values true, false, and nil.
- Symbols
- "Small integers." What's a small integer? If you are familiar with Ruby before 2.4, it's an integer small enough to be represented as a Fixnum. For everybody else, it's an integer small enough to fit in one memory location, so on a 64-bit machine, 2^^62 - 1. (That's one bit for sign and one bit to mark it as an integer.)
- Instances of type Float, Complex, Rational, String, or Regexp or larger Integers if they have been frozen.
- Instances of Class or Module – not instances of an individual class, instances of Class itself.
- Individual ractors.
- An instance of an object whose instance variables are all sharable.

Ruby provides the method Ractor.make_sharable(obj) which tries to make an arbitrary object sharable by walking through all its attributes and freezing them all. With the keyword argument copy: true it makes a copy of the object and returns the copy.

Sharable objects are shared when passed to a ractor, meaning that a reference to them is passed along and both the sender and the receiver are still able to access the object. Unsharable objects are copied, unless you pass move: true to either Ractor.send or Ractor.yield. If you move the unsharable object, then it's available to the new ractor but critically, it's no longer available to the Ractor that sent it, attempting to access that variable after it moves will raise an exception.